

Pemrograman Web Lanjut

SIB245007 | D-IV Sistem Informasi Bisnis

Pertemuan 2: Protokol HTTP dan Pola MVC

Siklus Request/Response & Arsitektur Aplikasi Web

Yang Akan Kamu Pelajari

1. Menjelaskan siklus **request/response** HTTP (method, status code, header) dan memetakannya ke potongan kode routing
2. Membandingkan pola **MVC**, **MVVM**, dan **Clean Architecture**, serta menjelaskan bagaimana Laravel mengimplementasikan MVC
3. Mengenali peran **route**, **controller**, dan **middleware** dalam menangani sebuah permintaan web

Slide ini membahas konsep. Langkah menulis route, controller, dan middleware secara praktis dibahas terpisah di luar slide ini.

Bagian 1

Analogi Kantor Pos

Sepucuk surat

- Masuk lewat loket penerimaan
- Petugas loket membaca alamat & jenis surat
- Diteruskan ke bagian yang tepat
- Petugas loket tidak membuka amplop atau memutuskan isi balasan

Request ke Simple POS

- Masuk lewat satu pintu
- Disortir berdasarkan alamat & jenisnya
- Diteruskan ke bagian yang tepat untuk diproses
- Penyortir tidak memutuskan isi jawaban

Kalau penyortiran keliru — mis. permintaan hapus produk diteruskan tanpa memeriksa apakah pengirimnya admin — aplikasi kehilangan kendali atas siapa yang boleh mengubah apa.

Anatomi Sebuah Request

Request: permintaan yang dikirim browser ke server, berisi method, alamat (URL), header, dan kadang data (form, JSON).

Route: aturan yang memetakan satu kombinasi method dan alamat URL ke kode yang akan menanganinya.

- Setiap request selalu membawa sebuah **method** yang menyatakan maksud permintaan
- Method inilah yang menentukan route mana yang cocok — bukan alamat URL saja

Method HTTP

Method	Maksud	Contoh pada Simple POS
GET	Meminta data, tanpa mengubah apa pun di server	Menampilkan halaman <code>/transactions</code>
POST	Mengirim data baru	Menyimpan transaksi kasir baru
PATCH	Mengubah sebagian data yang sudah ada	Memperbarui stok produk
DELETE	Menghapus data	Menghapus produk dari katalog

Aturan ketat: `GET` tidak boleh mengubah data di server. Melanggar aturan ini membuat perilaku aplikasi sulit ditebak — mis. me-refresh halaman tanpa sengaja menghapus data.

Anatomi Sebuah Response

Response: jawaban yang dikirim server kembali ke browser, berisi status code, header, dan isi (HTML, JSON, redirect).



Response selalu membawa **status code**, angka tiga digit yang menyatakan hasil permintaan secara ringkas — sebelum satu byte pun dari isinya dibaca.

Status Code

Kode	Arti	Contoh pada Simple POS
200	OK	Halaman <code>/transactions</code> berhasil ditampilkan
302	Redirect	Setelah <code>/pos</code> disimpan, diarahkan ke halaman detail
404	Not Found	Membuka <code>/transactions/9999</code> untuk ID yang tidak ada
422	Unprocessable Entity	Form transaksi dikirim dengan stok tidak mencukupi
500	Server Error	Kesalahan tak tertangani di sisi server

Pola 302: Kirim-lalu-Redirect



Setelah `POST /pos` berhasil menyimpan transaksi, server tidak langsung mengirim HTML sebagai jawaban — ia mengirim response 302 berisi header `Location` yang memerintahkan browser meminta ulang ke alamat lain.

- Pola ini berulang di hampir setiap fitur tulis-data
- Mencegah pengguna menekan **refresh** dan tanpa sadar mengirim data yang sama dua kali

Header: Metadata di Luar Isi

Header: metadata yang menyertai request maupun response, di luar isi utamanya.

- `Content-Type` — memberi tahu format isi: `text/html` , `application/json` , dst.
- `Authorization` — membawa token untuk permintaan yang butuh identitas pengirim (dibahas lebih dalam saat topik API)

Header dibaca sebelum isi diproses — server bisa menolak atau mengarahkan request hanya berdasarkan header, tanpa membuka isinya.

Melihat HTTP Secara Langsung

- Browser modern menyediakan **DevTools** → **tab Network** untuk melihat setiap request-response yang terjadi
- Setiap baris menampilkan: method, alamat, status code, dan waktu respons
- Cara paling langsung membuktikan bahwa "di balik setiap klik ada request HTTP yang nyata"

Konsep ini akan langsung kamu praktikkan: membuka DevTools dan mengamati request sungguhan dari route yang kamu buat sendiri.

Bagian 2

Masalah yang Dijawab MVC

- Menerima request dan mengirim response saja **belum cukup** untuk menjaga kode tetap rapi begitu aplikasi tumbuh
- Tanpa pemisahan tanggung jawab yang tegas, kode "menyortir permintaan", "mengambil data", dan "menyusun tampilan" bercampur dalam satu berkas
- Pola **MVC** (Model-View-Controller) menjawabnya dengan membagi tiga tanggung jawab secara tegas

Model — View — Controller

MVC: Model mengurus data, View mengurus tampilan, Controller mengurus alur permintaan di antara keduanya.

Kembali ke analogi kantor pos

- Controller = petugas loket yang menyortir
- Model = arsip berisi data sesungguhnya
- View = formulir balasan yang diserahkan ke pengirim

Aturan praktis menaruh kode

- Menyentuh data → Model
- Mengatur alur permintaan → Controller
- Menampilkan → View

Framework tidak memaksa aturan ini — query di dalam view tetap bisa jalan. Pola ini adalah disiplin yang dijaga penulis kodenya sendiri.

MVC di Struktur Folder Laravel

Folder	Peran MVC
<code>routes/web.php</code>	Controller — pendaftaran alamat URL
<code>app/Http/Controllers/</code>	Controller — kelas pemroses request
<code>app/Models/</code>	Model — representasi data & aturan bisnis
<code>resources/views/</code>	View — tampilan yang dilihat pengguna

Struktur folder ini bukan kebetulan — ia mewujudkan pola MVC secara konsisten sejak proyek pertama kali dibuat.

MVVM

MVVM (Model-View-ViewModel): menyisipkan ViewModel di antara Model dan View; ViewModel menyimpan state tampilan dan otomatis menyinkronkannya ke View lewat binding dua arah.

- Populer pada aplikasi antarmuka **reaktif** — tampilan berubah terus-menerus tanpa reload halaman
- Lebih relevan untuk framework frontend (mis. Vue) dibanding aplikasi server-rendered seperti Simple POS saat ini

Clean Architecture

Clean Architecture: mengisolasi aturan bisnis inti (use case) sepenuhnya dari framework yang dipakai, sehingga bisa diuji dan dipindah ke framework lain tanpa disentuh.

- Manfaat: aturan bisnis bisa diuji dan dipindah tanpa bergantung pada framework
- Ongkos: lapisan abstraksi tambahan yang perlu dirawat

Untuk aplikasi seskala Simple POS, isolasi seketat ini menambah abstraksi yang belum sepadan manfaatnya — MVC bawaan Laravel sudah cukup rapi.

Perbandingan Tiga Pola

Pola	Pemisahan Utama	Contoh Pemakaian
MVC	Model, View, Controller	Laravel (bawaan)
MVVM	Model, View, ViewModel	Aplikasi dengan binding dua arah antara tampilan & state
Clean Architecture	Lapisan use case terisolasi dari framework	Sistem berskala besar dengan banyak aturan bisnis

Route, Controller, dan Middleware

Controller: class berisi method-method penanganan request, dipanggil oleh route yang cocok.

Middleware: lapisan yang memeriksa atau mengubah request sebelum sampai ke controller, misalnya memeriksa apakah pengguna sudah login.

- Route = alamat + method → kode yang menanganinya
- Middleware berdiri **di antara** router dan controller — request harus lolos dulu sebelum diproses
- Bagian praktikum akan menulis ketiganya untuk endpoint Simple POS

Rangkuman

- Setiap request HTTP membawa **method** (GET / POST / PATCH / DELETE) dan setiap response membawa **status code** tiga digit yang menyatakan hasil sebelum isinya dibaca
- **MVC** memisahkan Model (data & aturan bisnis), View (tampilan), dan Controller (penyortir request); **MVVM** cocok untuk antarmuka reaktif, **Clean Architecture** cocok untuk sistem besar dengan aturan bisnis kompleks
- Laravel memetakan MVC langsung ke routes/ , app/Http/Controllers/ , app/Models/ , dan resources/views/
- **Route**, **Controller**, dan **Middleware** bekerja berurutan: request dicocokkan lewat route, disaring middleware, baru diproses controller

Referensi & Diskusi

Dokumentasi resmi Laravel · Disertasi Fielding (2000) tentang REST

Kode lengkap: `github.com/se-polinema/simple-pos`

Pertemuan berikutnya: Frontend & Templating (Blade, Tailwind, Alpine)